Uma **thread** é o menor fluxo de controle ou sequência de execução de instruções que o processador consegue gerenciar dentro de um programa. [1]

Aplicação Single-Thread (Thread Única)

- **O que é:** Um programa que possui apenas uma linha de execução. Todas as tarefas rodam de forma estritamente sequencial, ou seja, uma instrução acontece após a outra, na mesma ordem. [1]
- **Como funciona:** Se o programa estiver realizando um cálculo pesado ou esperando uma resposta (como uma consulta a um banco de dados ou leitura de arquivo), a aplicação inteira "congela" ou fica bloqueada até que aquela tarefa termine. [1]
- **Exemplo:** Um script simples de linha de comando que lê um arquivo, processa os dados linha por linha e imprime o resultado final.

Aplicação Multithread (Múltiplas Threads)

- **O que é:** Um programa que consegue criar e gerenciar várias threads executando fluxos de trabalho separados ao mesmo tempo. [1, 2]
- **Como funciona:** Em processadores com múltiplos núcleos (*multi-core*), essas threads podem rodar verdadeiramente em **paralelo**, ou o processador alterna rapidamente entre elas para dar a sensação de simultaneidade (concorrência). [1]
- **Vantagens:** Melhora o desempenho geral e mantém a aplicação responsiva (por exemplo: uma thread cuida da interface gráfica ou das requisições web, enquanto outras threads processam arquivos pesados em segundo plano). [1, 2]
- **Como criar em Java:** Você pode implementar threads estendendo a classe `Thread` ou implementando a interface `Runnable`, além de usar recursos avançados como o `ExecutorService`. [1, 2]
- **Cuidados necessários:** Quando várias threads acessam e alteram os mesmos dados ao mesmo tempo, surgem problemas de concorrência. Para evitar isso, usa-se o conceito de **Thread-Safe** e blocos sincronizados (`synchronized`). [1, 2]

Aqui está um exemplo prático demonstrando a diferença entre as duas abordagens em Java. Ambos os códigos simulam duas tarefas pesadas (como o download de dois arquivos grandes), onde cada tarefa demora 2 segundos para terminar.

---

1. Exemplo Single-Thread (Sequencial)

Neste modelo, o programa usa apenas a thread principal (`main`). A Tarefa 2 só começa **depois** que a Tarefa 1 terminar completamente.

java

```java
public class ExemploSingleThread {
    public static void main(String[] args) {
        System.out.println("--- Iniciando aplicação Single-Thread ---");
        long inicio = System.currentTimeMillis();

        // Executa a Tarefa 1
        executarTarefa("Tarefa 1");

        // Executa a Tarefa 2 (espera a 1 terminar)
        executarTarefa("Tarefa 2");

        long fim = System.currentTimeMillis();
        System.out.println("Tempo total de execução: " + (fim - inicio) / 1000
+ " segundos.");
    }

    private static void ejecutarTarefa(String nomeTarefa) {
        System.out.println(nomeTarefa + " iniciada na thread: " +
```

Use o código com cuidado.

Resultado no console:

> --- Iniciando aplicação Single-Thread ---
>
> Tarefa 1 iniciada na thread: main
>
> Tarefa 1 concluída!
>
> Tarefa 2 iniciada na thread: main
>
> Tarefa 2 concluída!
>
> **Tempo total de execução: 4 segundos.**

---

2. Exemplo Multithread (Paralelo/Concorrente)

Neste modelo, criamos duas novas threads. A Tarefa 1 e a Tarefa 2 rodam **ao mesmo tempo**, de forma independente da thread principal.

java

```java
public class ExemploMultithread {
    public static void main(String[] args) {
        System.out.println("--- Iniciando aplicação Multithread ---");
        long inicio = System.currentTimeMillis();

        // Criamos a Thread para a Tarefa 1 usando uma expressão Lambda
(Runnable)
        Thread thread1 = new Thread(() -> executarTarefa("Tarefa 1"));
```

```
        // Criamos a Thread para a Tarefa 2
        Thread thread2 = new Thread(() -> executarTarefa("Tarefa 2"));

        // Inicia as duas threads (elas passam a rodar em paralelo)
        thread1.start();
        thread2.start();

        try {
```

Use o código com cuidado.

Resultado no console:

> --- Iniciando aplicação Multithread ---
>
> Tarefa 1 iniciada na thread: Thread-0
>
> Tarefa 2 iniciada na thread: Thread-1
>
> *(Janela de 2 segundos onde ambas rodam juntas)*
>
> Tarefa 1 concluída!
>
> Tarefa 2 concluída!
>
> **Tempo total de execução: 2 segundos.**

🔍 Resumo da Diferença

- No **Single-Thread**, o tempo soma (**2s + 2s = 4s**) porque uma tarefa barra a outra.
- No **Multithread**, o tempo total é apenas o da tarefa mais longa (**2s**), pois o processador executa as duas simultaneamente em threads separadas (`Thread-0` e `Thread-1`).